In [4]:

```
# ================================================================================
# SecureBMS: OneBattery OneCell Full Parameter & Degradation Analyzer
# Compatible with Jupyter Notebook / Linux / Python 3.8+
# ================================================================================

import os
import glob
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --------------------------------------------------------------------------------
# 1. AUTO-DETECT & LOAD DATASET
# --------------------------------------------------------------------------------
TARGET_FILENAME = 'SecureBMS_OneBattery_OneCell_1000.csv'
CSV_PATH = None

# 1. Check direct and relative paths (including inside Zerova folder)
candidate_paths = [
    TARGET_FILENAME,
    os.path.join('Zerova', TARGET_FILENAME),
    os.path.expanduser(f'~/Zerova/{TARGET_FILENAME}'),
    os.path.expanduser(f'~/{TARGET_FILENAME}')
]

for path in candidate_paths:
    if os.path.exists(path):
        CSV_PATH = path
        break

# 2. If not found, search for any file starting with "SecureBMS_OneB" in current &
if CSV_PATH is None:
    search_patterns = [
        'SecureBMS_OneB*',
        'Zerova/SecureBMS_OneB*',
        '*/SecureBMS_OneB*',
        os.path.expanduser('~/Zerova/SecureBMS_OneB*')
    ]
    for pattern in search_patterns:
        matches = [f for f in glob.glob(pattern) if not f.endswith('.ipynb')]
        if matches:
            CSV_PATH = matches[0]
            break

if CSV_PATH is None:
    raise FileNotFoundError(
        f"Could not find '{TARGET_FILENAME}'.\n"
        f"Current Working Directory: {os.getcwd()}\n"
        f"Files in current folder: {os.listdir('.')}"
    )

print(f"Loaded dataset from: {os.path.abspath(CSV_PATH)}")
df = pd.read_csv(CSV_PATH)
```

```
print("=" * 80)
print(f" SECURE BMS ANALYSIS REPORT: {os.path.basename(CSV_PATH)}")
print("=" * 80)

# ------------------------------------------------------------------------------
# 2. VEHICLE TRIP DYNAMICS
# ------------------------------------------------------------------------------
duration_s = len(df)
duration_min = duration_s / 60.0
total_distance_km = df['km_travelled'].iloc[-1]
avg_speed = df['speed_kmph'].mean()
max_speed = df['speed_kmph'].max()
min_speed = df['speed_kmph'].min()

print("\n[1] VEHICLE DYNAMICS & TRIP PROFILE")
print(f"  • Total Duration       : {duration_s} seconds ({duration_min:.2f} minutes
print(f"  • Total Distance       : {total_distance_km:.3f} km")
print(f"  • Average Speed        : {avg_speed:.2f} km/h")
print(f"  • Speed Range          : {min_speed:.2f} km/h - {max_speed:.2f} km/h")

# ------------------------------------------------------------------------------
# 3. ELECTRICAL CHARACTERISTICS & CAPACITY ESTIMATION
# ------------------------------------------------------------------------------
soc_start = df['soc_pct'].iloc[0]
soc_end = df['soc_pct'].iloc[-1]
delta_soc = soc_start - soc_end

# Coulomb counting: Ah = sum(current * dt) / 3600
total_as = df['current_A'].sum()
total_ah = total_as / 3600.0
nominal_capacity_ah = total_ah / (delta_soc / 100.0)

avg_current = df['current_A'].mean()
max_current = df['current_A'].max()
min_current = df['current_A'].min()
avg_c_rate = avg_current / nominal_capacity_ah
peak_c_rate = max_current / nominal_capacity_ah

# Reconstruct Open Circuit Voltage (OCV): V_ocv = V_terminal + I * R_int
df['ocv_est_V'] = df['voltage_V'] + df['current_A'] * (df['internal_resistance_mohm

corr_raw_v_soc = df['voltage_V'].corr(df['soc_pct'])
corr_ocv_soc = df['ocv_est_V'].corr(df['soc_pct'])

print("\n[2] ELECTRICAL PARAMETERS & CELL SPECIFICATIONS")
print(f"  • State of Charge (SoC): Starts at {soc_start:.2f}%, Ends at {soc_end:.2f
print(f"  • Total Charge Drawn   : {total_ah:.4f} Ah ({total_as:.1f} A*s)")
print(f"  • Nominal Cell Capacity: {nominal_capacity_ah:.2f} Ah ({nominal_capacity_
print(f"  • Discharge Current    : Mean={avg_current:.2f} A, Max={max_current:.2f}
print(f"  • Discharge C-Rate     : Mean={avg_c_rate:.2f}C, Peak={peak_c_rate:.2f}C"
print(f"  • Terminal Voltage     : Mean={df['voltage_V'].mean():.4f} V, Range=[{df[
print(f"  • Reconstructed OCV    : Mean={df['ocv_est_V'].mean():.4f} V, Range=[{df[
print(f"  • Correlation (V vs SoC)  : {corr_raw_v_soc:+.4f} (Raw Terminal Voltage)"
print(f"  • Correlation (OCV vs SoC): {corr_ocv_soc:+.4f} (Ohmic Drop Compensated)"
```

```
# ------------------------------------------------------------------------------
# 4. THERMAL DYNAMICS & JOULE HEATING
# ------------------------------------------------------------------------------
temp_min = df['temperature_C'].min()
temp_max = df['temperature_C'].max()
temp_mean = df['temperature_C'].mean()
delta_temp = temp_max - temp_min

# Joule heating power: P_loss = I^2 * R
df['joule_power_W'] = (df['current_A'] ** 2) * (df['internal_resistance_mohm'] / 10
corr_temp_current = df['temperature_C'].corr(df['current_A'])
corr_temp_power = df['temperature_C'].corr(df['joule_power_W'])

print("\n[3] THERMAL DYNAMICS")
print(f"  • Temperature Range    : {temp_min:.2f} °C to {temp_max:.2f} °C (Delta: {
print(f"  • Mean Temperature     : {temp_mean:.2f} °C")
print(f"  • Correlation (T vs I) : {corr_temp_current:+.4f} (Strong Joule heating d
print(f"  • Mean Internal Heat   : {df['joule_power_W'].mean():.3f} W (Peak: {df['j

# ------------------------------------------------------------------------------
# 5. MECHANICAL STRESS & SPEED HUMP EVENTS
# ------------------------------------------------------------------------------
hump_indices = df[df['hump_event'] == 1].index.tolist()
hump_timestamps = df.loc[hump_indices, 'timestamp'].tolist()
num_humps = len(hump_indices)

vib_baseline = df[df['hump_event'] == 0]['vibration_g']
vib_humps = df[df['hump_event'] == 1]['vibration_g']
vib_spike_ratio = vib_humps.mean() / vib_baseline.mean()

print("\n[4] MECHANICAL STRESS & ROAD ANOMALIES")
print(f"  • Hump Events Count    : {num_humps} road humps encountered")
print(f"  • Hump Timestamps (s)  : {hump_timestamps}")
print(f"  • Baseline Vibration   : Mean={vib_baseline.mean():.4f} g (Max={vib_basel
print(f"  • Impact Vibration     : Mean={vib_humps.mean():.4f} g (Min={vib_humps.mi
print(f"  • Vibration Shock Ratio: {vib_spike_ratio:.1f}x higher during speed hump

# ------------------------------------------------------------------------------
# 6. ELECTRO-MECHANICAL DEGRADATION: SOH, MICROCRACKS & RESISTANCE
# ------------------------------------------------------------------------------
mc_start = df['microcrack_index'].iloc[0]
mc_end = df['microcrack_index'].iloc[-1]
mc_growth_pct = ((mc_end - mc_start) / mc_start) * 100.0

r_start = df['internal_resistance_mohm'].iloc[0]
r_end = df['internal_resistance_mohm'].iloc[-1]
r_growth_pct = ((r_end - r_start) / r_start) * 100.0

soh_start = df['soh_pct'].iloc[0]
soh_end = df['soh_pct'].iloc[-1]
total_soh_drop = soh_start - soh_end

# Decompose SoH loss: shock vs continuous
soh_diffs = df['soh_pct'].shift(1) - df['soh_pct']
hump_soh_loss = soh_diffs[df['hump_event'] == 1].sum()
normal_soh_loss = soh_diffs[df['hump_event'] == 0].sum()
```

```
pct_loss_shock = (hump_soh_loss / total_soh_drop) * 100.0
pct_loss_normal = (normal_soh_loss / total_soh_drop) * 100.0
avg_drop_per_hump = hump_soh_loss / num_humps
normal_rate_per_sec = normal_soh_loss / (len(df) - num_humps)

# Linear regression for R_int vs Microcracks
slope_r, intercept_r = np.polyfit(df['microcrack_index'], df['internal_resistance_m
r2_r = np.corrcoef(df['microcrack_index'], df['internal_resistance_mohm'])[0, 1] **

# Linear regression for SoH vs Microcracks
slope_soh, intercept_soh = np.polyfit(df['microcrack_index'], df['soh_pct'], 1)
r2_soh = np.corrcoef(df['microcrack_index'], df['soh_pct'])[0, 1] ** 2

print("\n[5] DEGRADATION & DAMAGE BREAKDOWN")
print(f"  • Microcrack Index     : {mc_start:.4f} -> {mc_end:.4f} (+{mc_growth_pct:
print(f"  • Internal Resistance  : {r_start:.2f} mOhm -> {r_end:.2f} mOhm (+{r_grow
print(f"  • Total SoH Loss       : {total_soh_drop:.4f}% ({soh_start:.4f}% -> {soh_
print(f"    - Shock Loss (Humps) : {hump_soh_loss:.4f}% ({pct_loss_shock:.1f}% of t
print(f"    - Cycling Loss       : {normal_soh_loss:.4f}% ({pct_loss_normal:.1f}% o
print(f"    - Avg Loss per Hump  : {avg_drop_per_hump:.6f}% per hump")
print(f"    - Normal Decay Rate  : {normal_rate_per_sec:.8f}% per second")
print(f"  • Resistance Model     : R_int = {intercept_r:.2f} + {slope_r:.2f} * Micr
print(f"  • SoH Damage Model     : SoH = {intercept_soh:.2f} - {abs(slope_soh):.2f}

# ------------------------------------------------------------------------------
# 7. CYBER-SECURITY / ANOMALY LABEL VERIFICATION
# ------------------------------------------------------------------------------
anomaly_counts = df['anomaly_label'].value_counts().to_dict()
print("\n[6] SECURITY & ANOMALY DETECTION (SecureBMS)")
print(f"  • Anomaly Label Counts : {anomaly_counts}")
if set(df['anomaly_label'].unique()) == {0}:
    print("  • Verification Status  : CLEAN BENIGN RUN. Zero cyber-attacks or anoma
    print("  • Operational Role     : Baseline reference profile for normal harsh-r

print("\n" + "=" * 80)
print(" ANALYSIS COMPLETE")
print("=" * 80)

# ------------------------------------------------------------------------------
# 8. JUPYTER NOTEBOOK VISUALIZATION DASHBOARD
# ------------------------------------------------------------------------------
fig, axs = plt.subplots(4, 1, figsize=(12, 16), sharex=True)
plt.subplots_adjust(hspace=0.25)

# Plot 1: Speed & Vibration (Impact Events)
ax1 = axs[0]
ax1.plot(df['timestamp'], df['speed_kmph'], color='tab:blue', label='Speed (km/h)',
ax1.set_ylabel('Speed (km/h)', color='tab:blue')
ax1.tick_params(axis='y', labelcolor='tab:blue')

ax1_twin = ax1.twinx()
ax1_twin.plot(df['timestamp'], df['vibration_g'], color='tab:red', label='Vibration
ax1_twin.scatter(df.loc[hump_indices, 'timestamp'], df.loc[hump_indices, 'vibration
                 color='darkred', s=40, zorder=5, label='Hump Shock')
ax1_twin.set_ylabel('Vibration (g)', color='tab:red')
```

```
ax1_twin.tick_params(axis='y', labelcolor='tab:red')
ax1.set_title('Vehicle Speed & Mechanical Vibration Spikes', fontweight='bold')
ax1.grid(True, alpha=0.3)

# Plot 2: Terminal Voltage & OCV vs SoC
ax2 = axs[1]
ax2.plot(df['timestamp'], df['voltage_V'], color='tab:purple', label='Terminal Volt
ax2.plot(df['timestamp'], df['ocv_est_V'], color='tab:orange', linestyle='--', labe
ax2.set_ylabel('Voltage (V)')
ax2.legend(loc='upper right')

ax2_twin = ax2.twinx()
ax2_twin.plot(df['timestamp'], df['soc_pct'], color='tab:green', label='SoC (%)')
ax2_twin.set_ylabel('State of Charge (%)', color='tab:green')
ax2_twin.tick_params(axis='y', labelcolor='tab:green')
ax2.set_title('Voltage Curves & State of Charge (SoC) Depletion', fontweight='bold'
ax2.grid(True, alpha=0.3)

# Plot 3: Current & Temperature
ax3 = axs[2]
ax3.plot(df['timestamp'], df['current_A'], color='tab:brown', label='Current (A)',
ax3.set_ylabel('Discharge Current (A)', color='tab:brown')
ax3.tick_params(axis='y', labelcolor='tab:brown')

ax3_twin = ax3.twinx()
ax3_twin.plot(df['timestamp'], df['temperature_C'], color='tab:olive', label='Tempe
ax3_twin.set_ylabel('Temperature (°C)', color='tab:olive')
ax3_twin.tick_params(axis='y', labelcolor='tab:olive')
ax3.set_title('Current Draw & Battery Cell Heating', fontweight='bold')
ax3.grid(True, alpha=0.3)

# Plot 4: Microcrack Index, Internal Resistance & SoH
ax4 = axs[3]
ax4.plot(df['timestamp'], df['internal_resistance_mohm'], color='darkorange', label
ax4.plot(df['timestamp'], df['microcrack_index'] * 100, color='crimson', linestyle=
ax4.set_ylabel('Resistance (mOhm) / Damage', color='darkorange')
ax4.tick_params(axis='y', labelcolor='darkorange')
ax4.legend(loc='center left')

ax4_twin = ax4.twinx()
ax4_twin.plot(df['timestamp'], df['soh_pct'], color='tab:blue', linewidth=2, label=
ax4_twin.set_ylabel('State of Health (%)', color='tab:blue')
ax4_twin.tick_params(axis='y', labelcolor='tab:blue')
ax4.set_title('Electro-Mechanical Degradation: Microcracks, Resistance Growth & SoH
ax4.set_xlabel('Timestamp (seconds)', fontweight='bold')
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
Loaded dataset from: /home/ibm-user/Zerova/SecureBMS_OneBattery_OneCell_1000.csv.xls
================================================================================
 SECURE BMS ANALYSIS REPORT: SecureBMS_OneBattery_OneCell_1000.csv.xls
================================================================================

[1] VEHICLE DYNAMICS & TRIP PROFILE
  • Total Duration       : 1000 seconds (16.67 minutes)
  • Total Distance       : 8.322 km
  • Average Speed        : 29.96 km/h
  • Speed Range          : 8.32 km/h - 55.92 km/h

[2] ELECTRICAL PARAMETERS & CELL SPECIFICATIONS
  • State of Charge (SoC): Starts at 89.99%, Ends at 72.48% (Drop: 17.51%)
  • Total Charge Drawn   : 0.8761 Ah (3153.8 A*s)
  • Nominal Cell Capacity: 5.00 Ah (5003 mAh)
  • Discharge Current    : Mean=3.15 A, Max=6.61 A, Min=0.10 A
  • Discharge C-Rate     : Mean=0.63C, Peak=1.32C
  • Terminal Voltage     : Mean=3.8718 V, Range=[3.6295 V, 4.0580 V]
  • Reconstructed OCV    : Mean=3.9742 V, Range=[3.8582 V, 4.0950 V]
  • Correlation (V vs SoC)  : +0.7879 (Raw Terminal Voltage)
  • Correlation (OCV vs SoC): +0.9909 (Ohmic Drop Compensated)

[3] THERMAL DYNAMICS
  • Temperature Range    : 27.64 °C to 28.97 °C (Delta: 1.33 °C)
  • Mean Temperature     : 28.26 °C
  • Correlation (T vs I) : +0.6695 (Strong Joule heating dependence)
  • Mean Internal Heat   : 0.401 W (Peak: 1.674 W)

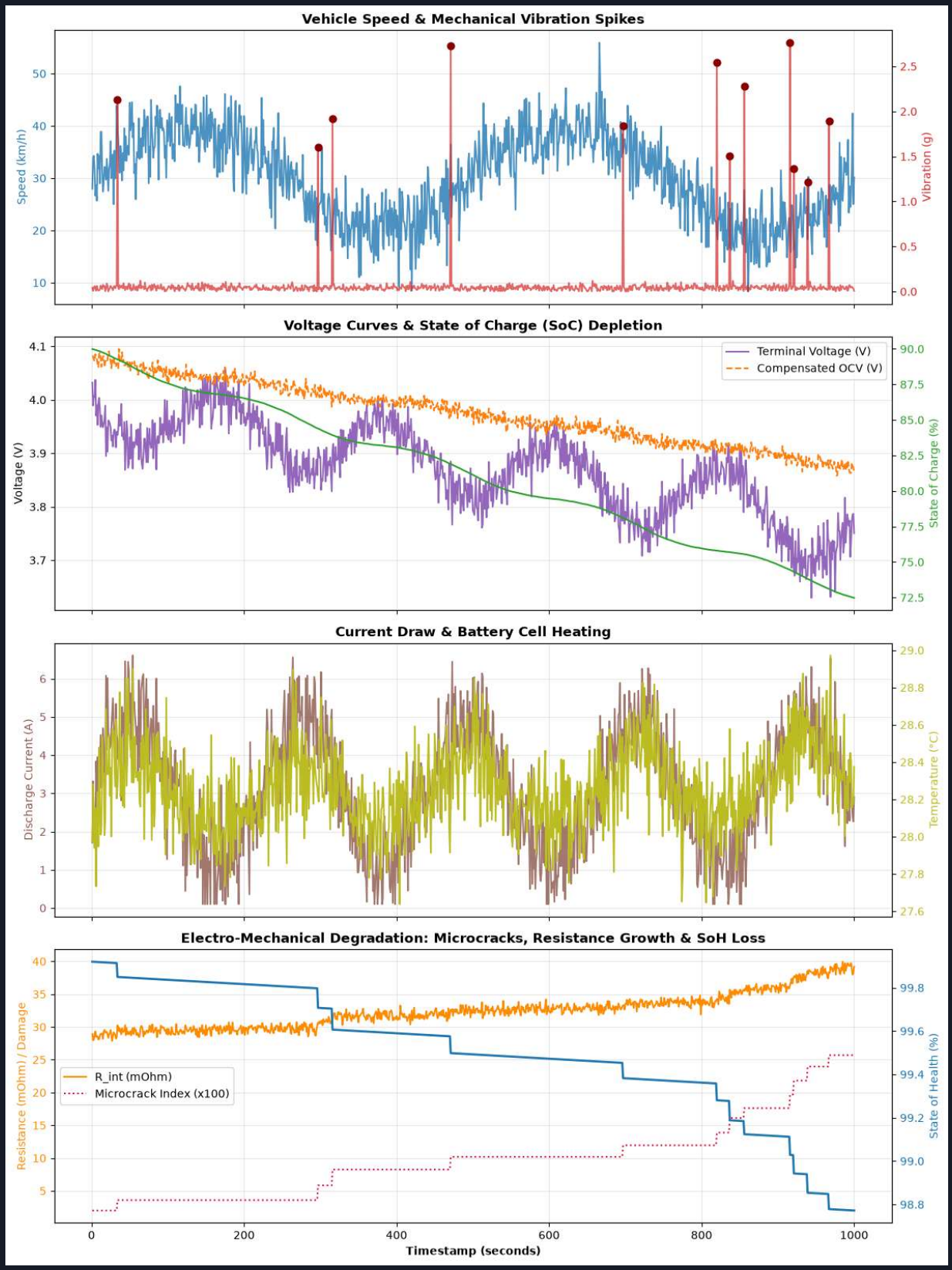
[4] MECHANICAL STRESS & ROAD ANOMALIES
  • Hump Events Count    : 12 road humps encountered
  • Hump Timestamps (s)  : [34, 297, 316, 471, 697, 820, 837, 856, 916, 921, 939, 96
7]
  • Baseline Vibration   : Mean=0.0414 g (Max=0.1206 g)
  • Impact Vibration     : Mean=1.9819 g (Min=1.2133 g, Max=2.7632 g)
  • Vibration Shock Ratio: 47.9x higher during speed hump impact

[5] DEGRADATION & DAMAGE BREAKDOWN
  • Microcrack Index     : 0.0200 -> 0.2570 (+1184.9% increase)
  • Internal Resistance  : 28.92 mOhm -> 39.19 mOhm (+35.5% increase)
  • Total SoH Loss       : 1.1477% (99.9198% -> 98.7721%)
    - Shock Loss (Humps) : 0.9503% (82.8% of total degradation)
    - Cycling Loss       : 0.1974% (17.2% of total degradation)
    - Avg Loss per Hump  : 0.079192% per hump
    - Normal Decay Rate  : 0.00019980% per second
  • Resistance Model     : R_int = 28.12 + 43.81 * Microcrack (R^2 = 0.960)
  • SoH Damage Model     : SoH = 99.99 - 4.90 * Microcrack (R^2 = 0.994)

[6] SECURITY & ANOMALY DETECTION (SecureBMS)
  • Anomaly Label Counts : {0: 1000}
  • Verification Status  : CLEAN BENIGN RUN. Zero cyber-attacks or anomalous sensor
faults.
  • Operational Role     : Baseline reference profile for normal harsh-road operatio
n.

================================================================================
```




Vehicle Speed & Mechanical Vibration Spikes
Speed (km/h)
Vibration (g)
Voltage Curves & State of Charge (SoC) Depletion
Terminal Voltage (V)
Compensated OCV (V)
Voltage (V)
State of Charge (%)
Current Draw & Battery Cell Heating
Discharge Current (A)
Temperature (°C)
Electro-Mechanical Degradation: Microcracks, Resistance Growth & SoH Loss
R_int (mOhm)
Microcrack Index (x100)
Resistance (mOhm) / Damage
State of Health (%)
Timestamp (seconds)